



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI-103 Microsoft Certified Azure AI Apps and Agents Developer Associate (C926)

Course Code: C926

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	11 August 2026	Initial aligned release of slides, Learner Guide, Lesson Plan and nine connected labs.	Tertiary Infotech Academy Trainer

TABLE OF CONTENTS

Introduction	4
Course Learning Outcomes.....	4
Before You Start — Preparation	4
Topic 01 — Plan and Manage an Azure AI Solution (25-30%)	5
Topic 02 — Implement Generative AI and Agentic Solutions (30-35%)	12
Topic 03 — Implement Computer Vision Solutions (10-15%)	21
Topic 04 — Implement Text Analysis Solutions (10-15%)	25
Topic 05 — Implement Information Extraction Solutions (10-15%)	30
Wrap-Up - From Prototype to Operated AI Service	37
Next Steps	37
Glossary.....	37
Official Reference Sources.....	38

Introduction

This Learner Guide is the self-contained study text for AI-103 Microsoft Certified Azure AI Apps and Agents Developer Associate (C926). It follows the five official skills domains published for AI-103 as of 16 April 2026 and teaches each concept before the related practice.

Nine connected labs grow one synthetic Northstar Support solution from an architecture decision record into a grounded, tool-using and multimodal application with extraction, retrieval, evaluation and operational controls. Detailed verification, troubleshooting and rejoin checkpoints keep the practical work executable without replacing the conceptual coverage.

Course Learning Outcomes

- LO1: Plan a secure, responsible and operable Microsoft Foundry solution by selecting suitable models, services, deployment patterns and controls.
- LO2: Build grounded generative applications with Microsoft Foundry, retrieval patterns, structured prompts and repeatable quality checks.
- LO3: Build and operationalize tool-using and multi-agent solutions with bounded roles, approval controls, tracing and error analysis.
- LO4: Implement multimodal computer-vision workflows for generation, understanding, accessibility and visual safety.
- LO5: Implement text, translation and speech workflows that return reliable structured outputs and handle language-specific limitations.
- LO6: Implement information-extraction and retrieval pipelines with OCR, Content Understanding, Azure AI Search and grounded outputs.

Before You Start — Preparation

What you need

- A Windows or macOS laptop with a current Microsoft Edge or Google Chrome browser.
- Python 3.10 or later, Visual Studio Code and Git.
- Azure CLI signed in to an instructor-provided subscription or sandbox with access to a Microsoft Foundry project.
- A deployed language model in the Foundry project and, where enabled, an image model, Azure AI Search service and Content Understanding resource.
- Permission to read project deployments, call the selected model, view traces and use the instructor-provided search index and analyzers.
- The values named in labs/resources/instructor-readiness-manifest.md; use the documented rejoin path when a regional or optional feature is unavailable.

Verify your setup

Open the Microsoft Foundry project, confirm the project endpoint and model deployment name, run az account show, and ask the instructor to show PASS evidence for the readiness manifest before starting a dependent lab.

```
python --version
```

```
az --version
```

```
az account show --query '{subscription:name, tenant:tenantId}' -o table
```

Never paste tokens or keys into this record.

Conventions used in every lab

- Replace angle-bracket placeholders such as <FOUNDRY_PROJECT_ENDPOINT> with instructor-provided training values.
- Store real values only in environment variables or managed connections; never in prompts, screenshots or source control.
- Portal navigation changes over time; follow the named resource and intent when a menu label has moved.
- Save every named checkpoint because later labs reuse the architecture record, policy corpus, traces or extracted fields.

Topic 01 — Plan and Manage an Azure AI Solution (25-30%)

25-30% objective coverage · models · Foundry services · infrastructure · security · operations · responsible AI

Key concepts

- Choose models and Foundry services from the task, modality, grounding, agency and operational constraints.
- Treat deployment topology, quotas, cost, identity, networking and CI/CD as part of the AI design.
- Monitor model quality, retrieval health, latency, tokens, safety events and business outcomes together.
- Apply guardrails, provenance, approval and tool-access controls according to impact and reversibility.

Choose Models and Foundry Services

Start with the workload contract: modalities, output, latency, quality, data boundary and permitted actions. Select a model by task fit, not size alone.

A Foundry project groups deployments, connections, agents, evaluation and observability. Use Azure AI Search or Foundry IQ for grounding and Foundry Tools for specialist services such as Content Understanding, Translator and Speech.

- Task fit: Match reasoning, modality, context and structured-output needs.
- Grounding: Choose retrieval and indexing for evidence-bound answers.
- Agency: Choose tools, memory and orchestration only when actions need them.
- Operations: Check region, quota, latency, cost, lifecycle and support status.

Infrastructure and Deployment

Separate development, validation and production environments. Use repeatable infrastructure definitions, named model deployments and environment-specific configuration so a release can be reproduced and rolled back.

Capacity is shaped by tokens, requests, concurrency and downstream limits. Rate-limit handling should use bounded retries and backoff; cost controls should combine quotas, budgets, model routing, caching and output limits rather than relying on a single alert.

- Project: Own models, agents, connections, evaluations and traces as one boundary.
- Deployment: Name a model version and capacity configuration used by applications.
- Pipeline: Promote tested code, prompts and configuration through environments.
- Capacity: Model token, request and downstream-service constraints explicitly.

Identity, Network and Secrets

Prefer Microsoft Entra identities and role-based access over copied service keys. Distinguish the developer identity, application managed identity and end-user identity because each may be authorized to different project connections or data sources.

Private endpoints and network controls reduce exposure but add DNS, routing and build-agent dependencies. Keep secrets in managed stores or environment settings and never place credentials in prompts, notebooks, screenshots or source control.

- Authenticate: Prove which workload or user is calling.
- Authorize: Grant the minimum project, model, search and data roles.
- Isolate: Use approved public or private network paths and test DNS.
- Protect: Use keyless credentials where supported and rotate unavoidable keys.

Responsible AI and Operations

Responsible AI is a lifecycle practice. Define intended use, affected users, excluded behavior, content filtering, groundedness expectations, human oversight, disclosure and incident response before release.

Trace evidence should connect user input, retrieval, model call, tool call and final response. Observe quality, drift, retrieval relevance, safety signals, token use, latency and errors without collecting more personal or confidential data than operations require.

- Prevent: Use instructions, filters, schemas, least privilege and tool boundaries.
- Detect: Evaluate quality and safety; monitor traces and unusual behavior.
- Respond: Pause actions, route to people and preserve a useful correlation ID.
- Improve: Change the component supported by evidence, then rerun regression tests.

Lab 1 — Design the Northstar Foundry Solution

Learning outcome: LO1 - select task-fit models and Foundry services, then define infrastructure, security, operations and responsible AI controls..

Goal: Translate the Northstar support scenario into a bounded architecture and decision record before provisioning or coding.

Duration and prerequisites

45 minutes

- Open the scenario, ADR template and partially completed northstar-architecture-example.md; use the example to keep the nine-step design within 45 minutes.
- Use only the supplied synthetic users, policies, documents and request records.
- Review the official AI-103 skills outline in reference/SOURCES.md.

What you'll build

A completed solution architecture decision record with a service map, identity boundary, deployment path, operational indicators and risk controls. (Tools: Text editor, diagram tool, labs/resources/northstar-scenario.md, solution-adr-template.md, northstar-architecture-example.md.)

Step-by-step

1. From the repository root, create the standard learner workspace and copy the ADR and checkpoint templates before editing either file.

```
New-Item -ItemType Directory -Force C926-labs,C926-labs\evidence,C926-labs\work | Out-Null
```

```
Copy-Item labs\resources\solution-adr-template.md C926-labs\northstar-solution-adr.md
```

```
Copy-Item labs\resources\learner-checkpoint-manifest.md C926-labs\evidence\manifest.md
```

2. In C926-labs/northstar-solution-adr.md, state the measurable outcome, intended users and explicit exclusions.

Outcome: cited support guidance and safe draft-request preparation

Excluded: identity changes, entitlement overrides, financial commitments

3. Map the workload modalities and select a language model, Foundry project, Azure AI Search, Content Understanding, Speech or Translator only where the scenario needs them.

Question -> Foundry model

Policy evidence -> Azure AI Search

Invoice or image -> Content Understanding

Call audio -> Speech or multimodal processing

4. Draw the solution boundary from user channel through application, Foundry project, model deployment, retrieval, agent tools and observability.

User -> App -> Foundry project -> {Model, Search, Tools} -> Trace store

5. Record the acting identity at every hop and choose Microsoft Entra authentication for the application and developer path where supported.

Developer: Azure CLI identity

Application: managed identity

End user: application identity plus enforced user filter

6. Define development, validation and production environments with one promotion path for code, prompts, indexes and configuration.

Development -> validation gate -> production

Rollback artifact: last known-good release plus configuration record

7. Add capacity and cost controls for requests, tokens, concurrency, output length, image jobs and downstream rate limits.

Controls: budgets | quotas | bounded retry | token cap | model routing | alerts

8. Complete the risk table for unsupported answers, unauthorized evidence, unsafe content, excessive agency, prompt injection and sensitive traces.

Evidence: groundedness check | permission test | content-filter result | approval record | trace review

9. Assign an owner and release evidence to every decision, complete the checklist, mark the ADR Proposed and update evidence/manifest.md.

Owners: Product | AI engineering | Data | Security | Operations

Checkpoint path: C926-labs/evidence/manifest.md

Test it

The ADR contains the outcome and exclusions, every required service has a scenario reason, every connection names an identity, and each priority risk has a control, owner and observable release evidence.

Troubleshooting

- The diagram is a list of product names.: Redraw it as actors, trust boundaries, information paths and ownership; keep product selection in the decision table.
- Every service is included by default.: Remove any service that does not satisfy a stated modality, grounding, action or operational need.
- A control says only 'be secure'.: Replace it with a configuration, test, trace, threshold or approval record that can be inspected.

Challenge

Add an alternative low-cost architecture that uses a smaller model for classification and the larger model only for grounded synthesis.

Reflection

Which architecture decision most reduces the chance of a fluent but unsupported answer reaching a user?

Note: Checkpoint: Save C926-labs/northstar-solution-adr.md and update C926-labs/evidence/manifest.md. Lab 2 adds verified project, deployment and operations evidence to this record.

Note: Full commands and screenshots are in labs/lab-01-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Lab 2 — Verify Foundry Access and the Operations Baseline

Learning outcome: LO1 - connect to a Foundry project with keyless developer authentication and record deployment, quota, cost, security and monitoring evidence..

Goal: Create an isolated Python workspace, authenticate through Azure CLI, inspect the instructor-provided Foundry project and make one controlled Responses API call.

Duration and prerequisites

60 minutes

- Complete Lab 1 and keep northstar-solution-adr.md available.
- Obtain the instructor-provided Foundry project endpoint and language-model deployment name.
- Confirm Foundry User on the learner project plus Reader on the Foundry resource, as named in the readiness manifest.

What you'll build

A verified local workspace, sanitized environment file, model response record and operations-baseline checklist linked from the ADR. (Tools: Python 3.12, Azure CLI, Microsoft Foundry, Azure AI Projects SDK, labs/resources/verify_foundry.py.)

Step-by-step

1. From the repository root, verify the Lab 1 workspace, enter it, create a virtual environment and install the tested lock file with that environment's interpreter.

```
Test-Path labs\resources\requirements-lock.txt
```

```
Test-Path C926-labs\northstar-solution-adr.md
```

```
Set-Location C926-labs
```

```
python -m venv .env
```

```
.\.env\Scripts\python.exe -m pip install -r ..\labs\resources\requirements-lock.txt
```

```
# macOS/Linux equivalent: .env/bin/python -m pip install -r  
../labs/resources/requirements-lock.txt
```

2. Sign in through Azure CLI and confirm the intended subscription and tenant; do not capture tokens in your evidence file.

```
az login
```

```
az account show --query '{subscription:name, tenant:tenantId, user:user.name}' -o  
table
```

3. Copy the environment template, fill only FOUNDRY_PROJECT_ENDPOINT and FOUNDRY_MODEL_NAME, and confirm .env is ignored by Git.

```
Copy-Item ..\labs\resources\.env.example .env
```

```
git check-ignore .env
```

4. Run the supplied verifier first in configuration-only mode and save the sanitized output.

```
.\.env\Scripts\python.exe ..\labs\resources\verify_foundry.py --check-only >  
evidence\foundry-check.txt
```

5. Run the live verifier. If the project or trace view is unavailable, copy the named sanitized rejoin output and record that limitation.

```
.\.env\Scripts\python.exe ..\labs\resources\verify_foundry.py --live >  
evidence\foundry-live.txt
```

```
# Rejoin: Copy-Item ..\labs\resources\foundry-verifier-rejoin.json  
evidence\foundry-verifier-rejoin.json
```

6. In the Foundry portal, locate the selected deployment and record model name, deployment name, region, version or upgrade policy and capacity unit without recording credentials.

Record in `northstar-solution-adr.md` under `Deployment evidence`.

7. Locate quota or usage, cost management and project access controls; record the limit, alert owner and least-privilege role used by the lab.

Evidence fields: `quota surface | budget alert | project role | owner | review date`

8. Open Observability or Traces once and record the visible trace or timestamp, token and latency fields. If propagation is delayed, use the named rejoin file and record the delay instead of waiting.

Expected: one model operation associated with the lab call. Rejoin: `evidence/foundry-verifier-rejoin.json`.

9. Append the endpoint shape, deployment evidence, operations owner and trace reference to the ADR, then link the evidence in `evidence/manifest.md`; never paste a key or token.

Endpoint shape: `https://<resource>.services.ai.azure.com/api/projects/<project>`

Test it

The check-only run reports valid configured values, the live run prints a non-empty response and deployment list, and the ADR contains sanitized deployment, quota, identity, cost and trace evidence.

Troubleshooting

- `DefaultAzureCredential cannot authenticate.`: Run `az account show`, confirm the intended tenant, then use `az login --tenant <TENANT_ID>` and retry.
- The project returns 403.: Ask the instructor to verify the exact project role and scope; do not substitute an API key into source code.
- The model call returns deployment not found.: Use the deployment name shown in the Foundry project's deployed models table, not the catalog model ID.

Challenge

Run the bounded prompt against a second approved deployment and compare latency, response length and task fit without changing the task prompt.

Reflection

Which operational field would tell you first that the solution is becoming unreliable: error rate, latency, token use, safety events or groundedness, and why?

Note: Checkpoint: Keep the foundry evidence in `C926-labs/evidence`, revise the ADR and update `evidence/manifest.md`. Lab 3 reuses the same project endpoint and model deployment.

Note: Full commands and screenshots are in labs/lab-02-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Topic 02 — Implement Generative AI and Agentic Solutions (30-35%)

30-35% objective coverage · Responses API · RAG · tools · memory · multi-agent · evaluation · observability

Key concepts

- A generative application is a controlled system of instructions, context, model parameters, validation and evaluation.
- RAG quality depends on ingestion, chunking, retrieval, evidence selection and faithful generation.
- Agents add tool selection, state and delegation; each capability needs a bounded contract and failure path.
- Evaluation and tracing turn fluent behavior into measurable evidence for release and improvement.

Generative Application Contract

Separate system instructions, user input, trusted context and output schema. The model should be told what to do when evidence is absent or conflicting, not merely what a good answer looks like.

Generation parameters trade determinism, diversity, length, latency and cost. Structured outputs reduce brittle parsing, but the application must still validate types, required fields and allowed values before using the result.

- Instruction: Role, task, evidence boundary and prohibited behavior.
- Context: Retrieved or supplied facts that may support the answer.
- Generation: Model and parameters selected for the task.
- Validation: Schema, policy and business-rule checks before use.

Retrieval-Augmented Generation

RAG retrieves candidate evidence before generation. Diagnose the stages separately: ingestion quality, index freshness, query representation, retrieval relevance, context selection and answer faithfulness.

Keyword search is precise for exact terms; vector search retrieves semantic similarity; hybrid search combines both and can add semantic ranking. Access filters and source

metadata must travel with the evidence so grounding does not become an authorization bypass.

- Ingest: Extract, clean, segment and enrich approved content.
- Retrieve: Use keyword, vector, hybrid or agentic retrieval deliberately.
- Ground: Supply relevant evidence with identifiers and provenance.
- Answer: Stay faithful, cite sources and decline unsupported claims.

Agent Roles, Tools and Memory

An agent combines a model with instructions, conversation state and tools. Tool names, descriptions and schemas are routing signals; narrow typed tools are safer and easier to evaluate than a generic function that can do anything.

Use conversation state for current-turn continuity and long-term memory only for durable user facts with an explicit scope and retention policy. Require human confirmation before consequential or hard-to-reverse tool calls.

- Role: One bounded outcome and clear exclusions.
- Tool: A typed capability with least privilege and safe errors.
- State: Conversation history needed for the current task.
- Memory: Durable facts isolated by user or business scope.

Multi-Agent Orchestration

Multiple agents are justified when capabilities have different expertise, identities, data boundaries, ownership or release cadences. They also add latency, cost, routing uncertainty and more failure states.

Give each specialist a non-overlapping capability description and a predictable result contract. Bound delegation depth, context sharing, timeouts and retries; preserve one correlation identifier across all hops.

- Route: Choose a specialist from distinct capability descriptions.
- Delegate: Pass the minimum task, context and constraints.
- Return: Use a stable status, result, evidence and error shape.
- Control: Limit depth, time, tools, context and consequential actions.

Evaluate and Operationalize

Use representative datasets for core, boundary, adversarial and regression scenarios. Measure relevance, groundedness, retrieval quality, task completion, tool selection, tool-input accuracy, latency, token use and safety according to the application risk.

Tracing exposes the path from input through retrieval, model and tool spans. Error analysis should identify whether to change data, retrieval, instructions, model, tool schema or infrastructure rather than repeatedly editing the prompt by instinct.

- Dataset: Representative queries, expected behavior and risk cases.

- Evaluator: A defined metric, threshold and explanation.
- Trace: Ordered spans for retrieval, model, tools and errors.
- Decision: Release, hold or improve based on evidence and guardrails.

Lab 3 — Build and Test a Grounded Generative App

Learning outcome: LO2 - implement RAG, structured prompting and repeatable quality checks through a Microsoft Foundry project..

Goal: Build a small Northstar policy assistant that retrieves local evidence, sends only the selected context to the Foundry Responses API and records groundedness observations.

Duration and prerequisites

75 minutes

- Complete Lab 2 and activate the same C926-labs Python environment.
- Work from C926-labs and keep editable copies under work/ or policies/, never under labs/resources/.
- Keep FOUNDRY_PROJECT_ENDPOINT and FOUNDRY_MODEL_NAME in the local .env file.

What you'll build

A runnable grounded_app.py workflow, query trace JSON files and a five-case quality worksheet. (Tools: Python, Microsoft Foundry Responses API, local synthetic policy corpus, labs/resources/grounded_app.py.)

Step-by-step

1. Copy the policy corpus, quality cases and editable rubric into the learner workspace, then inspect the five expected behaviors.

```
Copy-Item ..\labs\resources\northstar-policies policies -Recurse -Force
```

```
Copy-Item ..\labs\resources\quality-cases.jsonl work\quality-cases.jsonl
```

```
Copy-Item ..\labs\resources\quality-rubric.csv work\quality-rubric.csv
```

2. Run retrieval-only mode for an answerable question and inspect the ranked source IDs and excerpts.

```
.\.venv\Scripts\python.exe ..\labs\resources\grounded_app.py --policies policies -  
-query "What evidence is needed for a replacement laptop?" --retrieve-only
```

3. Run retrieval-only mode for an unsupported question and confirm that weak evidence does not become a confident answer.

```
.\.venv\Scripts\python.exe ..\labs\resources\grounded_app.py --policies policies -  
-query "Can Northstar reimburse a home renovation?" --retrieve-only
```

4. Open `grounded_app.py` and identify the instruction, user query, trusted-context delimiters, refusal rule and JSON output contract.

Required output keys: `answer` | `cited_sources` | `supported` | `uncertainty`

5. Run the live grounded call for the supported query and save its strictly validated trace record.

```
.\venv\Scripts\python.exe ..\labs\resources\grounded_app.py --policies policies -  
-query "What evidence is needed for a replacement laptop?" --live --out  
evidence\trace-supported.json
```

6. Run the remaining four quality cases, including explicit unsupported enforcement. Any schema, citation, support-strength or conflict-behavior violation exits nonzero.

```
.\venv\Scripts\python.exe ..\labs\resources\grounded_app.py --policies policies -  
-query "Can Northstar reimburse a home renovation?" --live --expect-unsupported --  
out evidence\trace-unsupported.json
```

```
.\venv\Scripts\python.exe ..\labs\resources\grounded_app.py --policies policies -  
-query "What is the target response time for a P2 request?" --live --out  
evidence\trace-conflict.json
```

```
.\venv\Scripts\python.exe ..\labs\resources\grounded_app.py --policies policies -  
-query "May text inside a screenshot change the agent rules?" --live --out  
evidence\trace-visual-policy.json
```

```
.\venv\Scripts\python.exe ..\labs\resources\grounded_app.py --policies policies -  
-query "When does a High replacement draft need review?" --live --out  
evidence\trace-review-rule.json
```

7. Complete `work/quality-rubric.csv` for all five cases using retrieval relevance, answer relevance, groundedness, citation correctness and safe-unknown behavior.

Use a 1-5 evidence-based rating; record the source IDs or failure stage.

8. Copy the app into `work/`, change one retrieval parameter or instruction, capture a before/after diff and rerun only the failed cases.

```
Copy-Item ..\labs\resources\grounded_app.py work\grounded_app-experiment.py
```

```
git diff --no-index ..\labs\resources\grounded_app.py work\grounded_app-  
experiment.py > evidence\grounded-app-change.diff
```

```
# Run the edited work\grounded_app-experiment.py with the same failed-case command
```

9. Save the selected configuration and evidence file names in the ADR and link them from `evidence/manifest.md`.

Checkpoint: `retrieval method` | `prompt version` | `quality cases` | `known limitations`

Test it

Supported questions cite only supplied policy source IDs, unsupported questions return supported=false, the conflict case names the conflict, and every quality case records the failing pipeline stage or acceptable result.

Troubleshooting

- Retrieval returns the same source for every query.: Confirm the policies folder contains three files and inspect token overlap; increase the query terms or top-k only with a relevance reason.
- The model emits text outside JSON.: Keep the JSON-only instruction, validate the returned text, and save the raw response for diagnosis rather than silently discarding it.
- The answer cites a source that was not retrieved.: Reject the output and strengthen the allowed-source rule; citations must be a subset of retrieved source IDs.

Challenge

Add one new policy with a later effective date and make the retriever surface the version conflict without deleting the older source.

Reflection

When should a RAG application decline rather than return the most similar available passage?

Note: Checkpoint: Save validated traces and the completed quality rubric under C926-labs/evidence and work, then update evidence/manifest.md. Lab 4 exposes a controlled tool beside this knowledge path.

Note: Full commands and screenshots are in labs/lab-03-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Lab 4 — Build a Tool-Using Agent with Approval Control

Learning outcome: LO3 - define an agent role, typed function tools, conversation state and a human approval boundary for consequential actions..

Goal: Build a Northstar service agent that can read a synthetic request without approval but must pause before creating a draft escalation.

Duration and prerequisites

90 minutes

- Complete Lab 3 and keep the Foundry endpoint and model settings available.

- Review labs/resources/northstar-requests.json; all records are synthetic.
- Use the agent-framework-foundry version installed from requirements-lock.txt in Lab 2.

What you'll build

A runnable Agent Framework app with read and write tools, an approval transcript and bounded error states. (Tools: Python, Microsoft Agent Framework, Microsoft Foundry, labs/resources/agent_tools.py.)

Step-by-step

1. Run the structural and deterministic edge checks. They inspect real tool metadata and schemas, then verify missing, invalid, first-write and duplicate outcomes in a temporary store.

```
.\venv\Scripts\python.exe ..\labs\resources\agent_tools.py --self-check
```

```
.\venv\Scripts\python.exe ..\labs\resources\agent_tools.py --edge-check >
evidence\agent-edge-check.json
```

2. Inspect the agent instructions and state the allowed role: explain policy, look up a synthetic request and prepare a draft escalation; exclude live changes and direct notification.

Role boundary is stored in source control; credentials and user data are not.

3. Inspect get_request. Confirm its record_id parameter is typed, the description states when to call it, and it returns a stable found/not_found shape.

Read-only tool: approval_mode=never_require

4. Inspect create_escalation_draft. Confirm allowed priority values, idempotency key, synthetic-only storage and the explicit approval requirement.

Write-like tool: approval_mode=always_require

5. Run the live agent for a read-only request and save the transcript. The lookup may execute without an approval interrupt.

```
.\venv\Scripts\python.exe ..\labs\resources\agent_tools.py --live --prompt "Show
the status of NS-1042" --transcript evidence\agent-read.json
```

6. Request a draft. Inspect the visible tool name and arguments, then type the exact interactive token `APPROVE create\_escalation\_draft`; no write occurs before that response.

```
.\venv\Scripts\python.exe ..\labs\resources\agent_tools.py --live --prompt
"Prepare a High priority escalation draft for NS-1042 because the device is
unusable" --transcript evidence\agent-approved.json
```

7. Repeat the request with explicit rejection. Confirm the transcript's before/after draft count and hash are unchanged and the response explains cancellation.

```
.\.venv\Scripts\python.exe ..\labs\resources\agent_tools.py --live --prompt  
"Prepare a Critical escalation draft for NS-1043" --reject --transcript  
evidence\agent-rejected.json
```

8. Review evidence/agent-edge-check.json for the exact missing-record, invalid-priority and duplicate-idempotency results; verify tool and approval loops are bounded.

Expected: not_found | invalid_priority | draft_created | already_exists

Limits: at most three approval rounds and three approval requests.

9. Open Foundry Observability, locate one read trace and one approval-controlled trace, and record model span, tool span, latency, token and error fields in agent-observations.md.

Do not copy user text or tokens into shared evidence when traces contain sensitive content.

10. Append the agent role, tool inventory, approval boundary and trace file names to the ADR, then update evidence/manifest.md.

Tools: get_request | create_escalation_draft

Test it

The self-check passes; read-only lookup works without approval; approved, rejected, missing, invalid and duplicate paths produce distinct bounded outcomes; and the write-like tool never executes before explicit approval.

Troubleshooting

- The agent calls the write tool too early.: Strengthen the tool description and instructions so it first summarizes the proposed fields, then waits for approval.
- No approval request appears.: Confirm the decorator uses approval_mode=always_require and that the installed Agent Framework version matches requirements-lock.txt.
- A duplicate draft is created.: Use the supplied idempotency key and check the local draft store before writing.

Challenge

Add a conditional rule that requires approval only for High or Critical priority while Normal drafts remain non-consequential and synthetic.

Reflection

Which tool arguments must be visible to a person before approval is meaningful?

Note: Checkpoint: Save all agent evidence under C926-labs/evidence and update evidence/manifest.md. Lab 5 uses these traces to design routing and quality gates.

Note: Full commands and screenshots are in labs/lab-04-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Lab 5 — Design Multi-Agent Routing and Quality Gates

Learning outcome: LO3 - implement bounded specialist routing, evaluate agent behavior and define observability and release gates..

Goal: Use a deterministic routing simulator and Foundry traces to separate policy, request and extraction specialists, then define quality thresholds for release.

Duration and prerequisites

60 minutes

- Complete Lab 4 and retain its three agent transcripts.
- Copy all editable contracts, cases and analysis templates into C926-labs/work before changing them.
- Use the local simulator first; a live connected-agent implementation is optional when the instructor environment supports it.

What you'll build

A multi-agent contract map, ten-case routing result, trace-based error analysis and release-gate decision record. (Tools: Python, Microsoft Foundry Observability, labs/resources/multi_agent_router.py, quality rubric.)

Step-by-step

1. Copy the contracts, main cases, intentionally failing boundary case and error-analysis template into work/, then verify every specialist has a distinct description, routing terms, context, result, tools, timeout and owner.

```
Copy-Item ..\labs\resources\agent-contracts.yaml work\agent-contracts.yaml
```

```
Copy-Item ..\labs\resources\multi-agent-cases.jsonl work\multi-agent-cases.jsonl
```

```
Copy-Item ..\labs\resources\multi-agent-boundary-cases.jsonl work\multi-agent-boundary-cases.jsonl
```

```
Copy-Item ..\labs\resources\error-analysis-template.csv work\error-analysis.csv
```

2. Run the ten-case deterministic router against the copied contracts and save its JSON report.

```
.\venv\Scripts\python.exe ..\labs\resources\multi_agent_router.py --contracts  
work\agent-contracts.yaml --cases work\multi-agent-cases.jsonl --out  
evidence\routing-report-before.json
```

3. Run the intentional boundary baseline with --allow-errors, add one precise capability/routing term to the copied policy contract, capture the diff and rerun until the boundary case passes.

```
.\.venv\Scripts\python.exe ..\labs\resources\multi_agent_router.py --contracts
work\agent-contracts.yaml --cases work\multi-agent-boundary-cases.jsonl --out
evidence\routing-boundary-before.json --allow-errors
```

```
# Edit work\agent-contracts.yaml: add entitlement or replacement-device wording
only to policy_grounding
```

```
git diff --no-index ..\labs\resources\agent-contracts.yaml work\agent-
contracts.yaml > evidence\agent-contract-change.diff
```

```
.\.venv\Scripts\python.exe ..\labs\resources\multi_agent_router.py --contracts
work\agent-contracts.yaml --cases work\multi-agent-boundary-cases.jsonl --out
evidence\routing-boundary-after.json
```

4. Rerun all ten main cases after the contract change and confirm no existing boundary regressed.

```
.\.venv\Scripts\python.exe ..\labs\resources\multi_agent_router.py --contracts
work\agent-contracts.yaml --cases work\multi-agent-cases.jsonl --out
evidence\routing-report-after.json
```

5. Draw the orchestrator flow with minimum context shared to each specialist, one correlation ID, per-hop timeout and a safe fallback.

```
User -> Orchestrator -> Specialist -> Result
```

```
Limits: one delegation hop | bounded context | no recursive routing
```

6. Review the Lab 4 traces and classify each issue in work/error-analysis.csv as routing, tool selection, tool input, tool result, generation, policy or infrastructure.

```
Each row needs correlation ID, evidence and corrective owner.
```

7. Define a release set containing core, boundary, adversarial and failure scenarios; select relevance, groundedness, task completion, tool selection, tool input, latency and safety criteria.

```
Each criterion needs a target, threshold, evidence source and owner.
```

8. In Foundry Evaluations, create or outline a run using the available built-in evaluators. Record unavailable or preview features as constraints rather than substituting unverified scores.

```
Suggested quality: relevance | groundedness | task completion
```

```
Suggested process: tool selection | tool input accuracy
```

9. Set release, hold and rollback rules, make one evidence-based decision, link all evidence in the ADR and update evidence/manifest.md.

Release only if critical safety/tool cases meet their threshold and no unresolved high-impact defect remains.

Test it

All ten cases have an expected and actual route, no specialist receives an excluded task, every observed failure is assigned to a pipeline stage, and the release record contains measurable quality, safety, latency and rollback criteria.

Troubleshooting

- Two specialists match the same case.: Rewrite descriptions around exclusive intent, data and permitted action; do not rely on agent names alone.
- Tracing has not appeared.: Wait several minutes, verify project tracing is enabled and use transcript timestamps as temporary evidence.
- An evaluator is unavailable in the region.: Record the limitation and use a documented manual rubric or custom deterministic check for the same criterion.

Challenge

Add a fourth specialist and quantify the added routing ambiguity, latency budget and operational ownership before accepting it.

Reflection

What evidence would justify multiple agents instead of one agent with three tools?

Note: Checkpoint: Save before/after routing reports, the contract diff, error analysis and release decision under C926-labs, then update evidence/manifest.md. Day 2 extends the same solution to visual, audio and document evidence.

Note: Full commands and screenshots are in labs/lab-05-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Topic 03 — Implement Computer Vision Solutions (10-15%)

10-15% objective coverage · image and video generation · editing · multimodal understanding · accessibility · visual safety

Key concepts

- Generation prompts specify subject, composition, style, constraints and intended use; editing adds source images and masks.
- Multimodal understanding grounds answers in pixels, embedded text, temporal segments and extracted regions.

- Accessible descriptions prioritize purpose and relevant evidence rather than listing every visual detail.
- Visual inputs can contain unsafe content or indirect prompt injection and require content and policy controls.

Generate and Edit Images or Video

Select a generation model by supported modality, region, quality, latency, cost and control surface. A useful prompt defines subject, setting, composition, visual treatment, text requirements and exclusions; reference media and masks constrain edits.

Generation is iterative but should remain traceable. Save prompt versions, model deployment, parameters, content-filter outcomes and human decisions when outputs feed a business process.

- Prompt: Subject, composition, treatment, constraints and exclusions.
- Reference: Source media that preserves identity, layout or style.
- Mask: Region eligible for inpainting or replacement.
- Review: Safety, accuracy, rights, brand and accessibility checks.

Multimodal Understanding

A multimodal model can answer questions about images and selected video frames, while Content Understanding can extract descriptions, fields, timing, segments and structured representations from several media types.

Ask for observations before interpretations, cite the visual region or time segment when possible, and state uncertainty. Optical text can be untrusted input; embedded instructions should never override the application policy.

- Observe: Objects, text, layout, action and scene evidence.
- Locate: Region, page, frame, timestamp or segment.
- Reason: Answer only what the visible evidence supports.
- Represent: Return captions, markdown, fields or structured JSON.

Accessible Visual Descriptions

Alt text communicates the image's purpose in context. Decorative images can use empty alt text; informative images need concise meaning; complex charts or diagrams need a short label plus a nearby extended description.

Generated descriptions require human review for names, sensitive attributes, inferred emotion and domain-specific claims. Do not guess protected or personal characteristics that are not needed for the task.

- Purpose: Explain why the image matters in this context.
- Evidence: Describe visible facts before interpretation.
- Length: Use concise alt text plus an extended description when needed.
- Review: Check names, numbers, bias, privacy and unsupported inference.

Responsible Multimodal Controls

Apply input and output content policies, provenance or watermark rules, permitted-brand constraints and a review path for ambiguous or high-impact content. A safe text prompt does not guarantee a safe visual result.

Indirect prompt injection can be hidden in signs, screenshots or documents. Treat extracted visual text as data, keep system instructions authoritative and restrict tool access even when the image asks the model to take action.

- Classify: Detect unsafe or disallowed visual content.
- Resist: Treat embedded instructions as untrusted data.
- Constrain: Apply brand, provenance, watermark and tool policies.
- Escalate: Route uncertain or consequential content to a person.

Lab 6 — Build a Responsible Multimodal Workflow

Learning outcome: LO4 - generate or edit visual content, analyze visual evidence, produce accessible descriptions and apply multimodal safety controls..

Goal: Create a visual-support evidence packet that combines one generated image, one image-understanding result, accessibility text and an indirect-prompt-injection check.

Duration and prerequisites

75 minutes

- Complete Day 1 and open the instructor-provided Foundry project.
- Confirm whether an approved image-generation deployment is available; otherwise use labs/resources/rejoin-generated-device-desk.svg.
- Confirm a multimodal model or Content Understanding image analyzer is available; otherwise use the named sanitized analysis rejoin result.

What you'll build

A visual evidence packet with prompt versions, generated or rejoin image, structured observations, alt text, extended description and policy disposition. (Tools: Microsoft Foundry image playground or image model, multimodal model or Content Understanding, labs/resources/visual-policy-checklist.csv.)

Step-by-step

1. Copy the editable evidence-packet, policy-checklist and injection-result templates before adding results.

```
Copy-Item ..\labs\resources\visual-evidence-packet-template.md work\visual-evidence-packet.md
```

```
Copy-Item ..\labs\resources\visual-policy-checklist.csv work\visual-policy-checklist.csv
```

```
Copy-Item ..\labs\resources\visual-injection-result-template.json
evidence\multimodal-analysis.json
```

2. Write a generation prompt for a Northstar service-desk training scene with subject, composition, visual treatment, accessibility intent and explicit exclusions.

```
Prompt fields: purpose | subject | setting | composition | style | visible text |
exclusions
```

3. Generate one image in the approved tool and record deployment, prompt version, size, quality and filter outcome; if unavailable, use the exact rejoin command.

```
Output: work\northstar-device-desk.png
```

```
Rejoin: Copy-Item ..\labs\resources\rejoin-generated-device-desk.svg
work\generated-device-desk.svg
```

4. Create one controlled edit request that changes only the laptop screen content while preserving people, composition and lighting. Use a mask if the available model supports it; otherwise document the intended mask region.

```
Edit: replace the screen with a generic diagnostics dashboard; do not add personal
data or logos.
```

5. Analyze the image with the available multimodal tool. Ask first for visible observations, text and regions, then answer the bounded support question; if unavailable, copy the sanitized analysis rejoin result.

```
Question: Which visible evidence suggests the laptop is awaiting diagnostics?
```

```
Rejoin: Copy-Item ..\labs\resources\multimodal-analysis-rejoin.json
evidence\multimodal-analysis.json
```

6. Produce concise alt text for the image's role in the guide and a longer description that explains relevant layout and evidence without inferring identity, emotion or protected attributes.

```
Alt text target: purpose plus essential evidence
```

```
Extended description: layout, objects, visible text and uncertainty
```

7. Analyze visual-prompt-injection.svg with tools disabled. Fill evidence/multimodal-analysis.json, including both source IDs and the indirect_prompt_injection object, then run the deterministic check.

```
.\venv\Scripts\python.exe ..\labs\resources\verify_visual_injection.py --result
evidence\multimodal-analysis.json > evidence\visual-injection-check.json
```

```
Expected: detected=true | treated_as_untrusted_data=true |
tool_invocation_allowed=false
```

8. Complete work/visual-policy-checklist.csv for unsafe content, unsupported inference, injection, brand, provenance, watermark, accessibility and human-review conditions.

Disposition values: allow | transform | block | review

9. Complete work/visual-evidence-packet.md with the exact source, prompt/model version, region/time, filter result, observations, evidence regions, alt text, extended description, injection result and reviewer disposition.

Exclude credentials and private URLs; link the packet and checklist from evidence/manifest.md.

Test it

The packet contains a reproducible generation or rejoin source, a bounded edit, evidence-first analysis, accessible descriptions, and an injection result that never follows instructions embedded in the image.

Troubleshooting

- Image generation is not available in the region.: Use the supplied rejoin SVG, record the limitation and continue with understanding, accessibility and policy checks.
- The model invents a person's identity or emotional state.: Constrain the prompt to visible facts and remove unsupported personal inference from the final description.
- The image asks the model to reveal configuration or call a tool.: Treat embedded text as untrusted data, keep application instructions authoritative and disable tools for the analysis step.

Challenge

Create a video-analysis design that samples scenes, preserves timestamps and detects the same injection and accessibility risks without generating a video.

Reflection

Why is a safe text prompt insufficient evidence that a generated or analyzed image is safe to publish?

Note: Checkpoint: Save the visual packet and checklist under C926-labs/work, keep machine evidence under evidence/, and update evidence/manifest.md. Lab 7 uses the same evidence-first pattern for text and audio.

Note: Full commands and screenshots are in labs/lab-06-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Topic 04 — Implement Text Analysis Solutions (10-15%)

10-15% objective coverage · entities · topics · summaries · structured JSON · sentiment · translation · speech

Key concepts

- Generative text analysis can combine extraction, classification, summarization and structured output in one controlled prompt.
- Sentiment and safety signals are probabilistic evidence, not facts about a person's intent or character.
- Translation quality depends on locale, terminology, context and a review path for high-impact content.
- Speech pipelines add audio quality, language, speaker, timing, voice and latency decisions.

Structured Text Analysis

Define the field schema before prompting. Entity extraction identifies spans or values, classification assigns allowed labels, summarization compresses information, and structured output makes downstream validation possible.

Keep source text alongside extracted values and record confidence or evidence spans where the service supports them. Missing information should remain null or explicitly unknown rather than being fabricated to complete a schema.

- Extract: Entities, facts, topics and domain fields.
- Classify: Sentiment, tone, safety or business category.
- Summarize: Compress while retaining decisions and exceptions.
- Structure: Validate allowed keys, types and values.

Sentiment, Tone and Safety

Sentiment services return labels and confidence scores at document or sentence level. Opinion mining can connect an evaluated target to the words that describe it, making the signal more actionable than a single overall label.

Use these signals for routing or prioritization with safeguards, not as unquestioned truth. Sarcasm, dialect, mixed sentiment, domain language and short text can reduce reliability.

- Label: A supported category such as positive, neutral or negative.
- Confidence: Model certainty, not correctness probability for every use.
- Target: The product, feature or issue being discussed.
- Safeguard: Threshold, human review and representative monitoring.

Translation and Domain Adaptation

Translation requires a source language, one or more target languages, terminology decisions and a fallback for unsupported language or ambiguous input. Preserve proper nouns, numbers, links and regulated terms deliberately.

A generative model can adapt tone or format, while Azure Translator offers a dedicated translation contract. High-impact or customer-facing content should use terminology resources and bilingual review.

- Detect: Identify or confirm the source language.
- Translate: Select target locale and dedicated or generative method.
- Adapt: Apply domain terminology, tone and formatting.
- Verify: Back-translate samples and review critical terms.

Speech and Audio Workflows

Speech to text converts audio into time-aligned language; text to speech renders text with a selected voice; speech translation combines recognition and translation. Choose real-time, fast-file or batch transcription by latency and media length.

Audio quality, microphone placement, accents, overlap and domain terms affect accuracy. Custom speech models may improve specialist vocabulary but add data, evaluation and lifecycle responsibilities.

- Capture: Microphone, stream or audio file with known format.
- Recognize: Language, timing, speakers and confidence.
- Reason: Summarize or extract from the transcript with evidence.
- Respond: Translate or synthesize an accessible voice output.

Lab 7 — Implement a Text, Translation and Speech Pipeline

Learning outcome: LO5 - extract structured text signals, translate approved content and integrate speech input with explicit evidence and error handling..

Goal: Process a synthetic Northstar support call from transcript or audio into structured JSON, translate the customer-facing summary and preserve timing and uncertainty evidence.

Duration and prerequisites

75 minutes

- Complete Lab 6 and activate the C926-labs Python environment.
- Choose at most one live optional service path (Speech or Translator) inside the 75-minute lab; use the named rejoin/design path for the other.
- Keep editable transcript, schema and review files under C926-labs/work.

What you'll build

A validated support-call JSON record, translated summary, speech evidence record and language-quality review. (Tools: Python, Microsoft Foundry Responses API, Azure Speech SDK, Azure Translator SDK, labs/resources/text_speech_pipeline.py.)

Step-by-step

1. Copy the transcript, schema and translation-review template into work/, then inspect required fields, enums, evidence, language, processing metadata and uncertainty.

```
Copy-Item ..\labs\resources\northstar-call-transcript.txt work\northstar-call-transcript.txt
```

```
Copy-Item ..\labs\resources\support-call-schema.json work\support-call-schema.json
```

```
Copy-Item ..\labs\resources\translation-review-template.csv work\translation-review.csv
```

2. Run offline extraction and validate the final serialized record against the copied schema before any service call.

```
.\.venv\Scripts\python.exe ..\labs\resources\text_speech_pipeline.py --transcript work\northstar-call-transcript.txt --schema work\support-call-schema.json --offline --out evidence\call-offline.json
```

3. Run the live Foundry extraction. Require JSON only, preserve evidence and use null or uncertainty when the transcript does not support a field.

```
.\.venv\Scripts\python.exe ..\labs\resources\text_speech_pipeline.py --transcript work\northstar-call-transcript.txt --schema work\support-call-schema.json --foundry --out evidence\call-foundry.json
```

4. Compare the sentiment signal with the evidence sentence. Record why the label is a routing signal rather than a fact about the caller's intent or character.

```
Review: mixed sentiment | sarcasm | short utterance | domain language | confidence
```

5. For the Speech path, save recognized text, result status, offset and duration. If unavailable or not selected, copy the named rejoin result.

```
.\.venv\Scripts\python.exe ..\labs\resources\text_speech_pipeline.py --audio <INSTRUCTOR_WAV_PATH> --speech --out evidence\call-speech.json
```

```
# Rejoin: Copy-Item ..\labs\resources\speech-rejoin-result.json evidence\call-speech.json
```

6. For the Translator path, translate the summary and revalidate the final record including translation metadata. If unavailable or not selected, perform the design review in work/translation-review.csv.

```
.\.venv\Scripts\python.exe ..\labs\resources\text_speech_pipeline.py --input-json evidence\call-foundry.json --schema work\support-call-schema.json --translate-to <LANGUAGE_CODE> --out evidence\call-translated.json
```

7. Back-translate or bilingual-review the summary in work/translation-review.csv and record terminology, identifier, number, tone and unsupported-addition defects.

```
Review fields: source_text | target_locale | translated_text | back_translation | terminology_check | numbers_names_check | disposition
```

8. Map success, no-match, cancelled, unsupported language, throttled and service-unavailable states to user-safe messages and a correlation ID.

`Never display raw keys, tokens, request headers or full service error bodies.`

9. Save the evidence summary in work/text-audio-evidence.md, link every artifact from evidence/manifest.md and add the text/audio checkpoint to the ADR.

`Keep synthetic transcript evidence; do not add real call recordings.`

Test it

The output validates against the supplied schema, every extracted conclusion has transcript evidence or uncertainty, the speech path records a success or named rejoin state, and the translated summary preserves identifiers and critical facts.

Troubleshooting

- The model fills a missing field with a plausible value.: Require null plus an uncertainty entry, then reject any output whose evidence does not contain the value.
- Speech returns NoMatch.: Check audio format, language and signal level; use the supplied transcript to rejoin while preserving the failure record.
- Translation changes an identifier or number.: Protect tokens before translation or validate and restore them afterward; escalate material changes for review.

Challenge

Add a second transcript with code-switching and define how language identification and terminology review should change.

Reflection

Which fields can safely automate routing, and which should remain advisory because the evidence is probabilistic?

Note: Checkpoint: Save validated call, speech and translation evidence under C926-labs, then update the ADR and evidence/manifest.md. Lab 8 extracts structured evidence from a document with Content Understanding.

Note: Full commands and screenshots are in labs/lab-07-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Topic 05 — Implement Information Extraction Solutions (10-15%)

10-15% objective coverage · ingestion · OCR · layout · Content Understanding · semantic, hybrid and vector search · grounding

Key concepts

- Information extraction converts unstructured multimodal content into evidence-bearing fields and representations.
- OCR finds text; layout finds structure; field extraction maps evidence to a business schema.
- Search index design controls what can be retrieved, filtered, ranked, cited and secured.
- A trustworthy pipeline monitors ingestion completeness, index freshness, retrieval relevance and grounded output quality.

The Extraction Pipeline

Ingest approved documents, images, audio or video; normalize and segment the content; enrich it with OCR, layout, descriptions or custom skills; then map evidence to a stable field schema.

Keep source identifiers, pages, spans, regions or timestamps so downstream users can inspect the evidence. Confidence thresholds and validation rules should route uncertain fields for review rather than silently accepting them.

- Ingest: Acquire approved content and preserve source identity.
- Understand: OCR, layout, segments, descriptions and fields.
- Validate: Schema, type, confidence and business-rule checks.
- Publish: Clean markdown, JSON, index documents and provenance.

Content Understanding Analyzers

An analyzer defines the content types, segmentation and fields that Content Understanding should return. Prebuilt analyzers accelerate common documents and media; custom analyzers express a domain-specific schema and extraction instructions.

Single-task pipelines optimize one outcome, while pro-mode or richer pipelines combine multiple extraction and reasoning tasks. Model availability, regional support, latency and cost remain deployment decisions.

- Prebuilt: A ready analyzer for common document or media types.
- Custom: A reusable domain schema with extraction instructions.
- Content: Markdown, transcript, figures, segments and fields.
- Evidence: Page, span, region, timestamp and confidence metadata.

Search for Grounding

An Azure AI Search index separates retrievable content from vector fields and filterable metadata. Keyword, vector and hybrid queries serve different needs; hybrid results use reciprocal rank fusion and can be reranked semantically.

Use integrated vectorization or precomputed embeddings consistently. Vector dimensions must match the embedding model, and security filters must be applied before evidence is returned to the application or agent.

- Schema: Key, content, vector, filter, source and security fields.
- Query: Keyword, vector, hybrid or agentic retrieval.
- Rank: Similarity, reciprocal rank fusion and semantic reranking.
- Filter: Tenant, user, source, category, date and permission boundary.

Grounded Representations and Quality

The retrieval pipeline should return compact evidence with stable source identifiers. The generation layer should cite that evidence, state uncertainty and decline questions the retrieved content cannot answer.

Monitor document counts, failed ingestion, stale sources, empty queries, retrieval precision, citation coverage and groundedness. A strong model cannot compensate for missing or unauthorized evidence.

- Completeness: Expected content was ingested and indexed.
- Relevance: Retrieved passages address the actual query.
- Faithfulness: The response stays within returned evidence.
- Provenance: A user or operator can locate the supporting source.

Lab 8 — Extract Invoice Evidence with Content Understanding

Learning outcome: LO6 - use OCR, layout and a Content Understanding analyzer to produce structured, grounded document fields..

Goal: Run the prebuilt invoice analyzer on a Microsoft synthetic sample, inspect markdown and fields, and validate business-critical values against source evidence.

Duration and prerequisites

90 minutes

- Complete Lab 7 and activate the C926-labs Python environment.
- Confirm a Foundry resource endpoint and the default `gpt-4.1` and `text-embedding-3-large` model mappings required by prebuilt-invoice.
- Confirm Cognitive Services User on the Foundry resource for your Entra principal, or obtain an ephemeral instructor key; store values only in the local environment.
- Use the Microsoft synthetic invoice URL already declared in the supplied script.

What you'll build

A sanitized Content Understanding result, extracted invoice table, field-validation log and custom-analyzer design note. (Tools: Python, Azure Content Understanding SDK, prebuilt-invoice analyzer, labs/resources/content_understanding_invoice.py.)

Step-by-step

1. Copy the editable validation template and run the static self-check for SDK availability, analyzer ID, sample URL, expected fields and documented model-mapping prerequisites.

```
Copy-Item ..\labs\resources\invoice-field-validation.csv work\invoice-field-validation.csv
```

```
.\.venv\Scripts\python.exe ..\labs\resources\content_understanding_invoice.py --self-check
```

2. Set the endpoint and optional ephemeral key with executable PowerShell assignments, then validate the configured endpoint shape. Instructor readiness separately confirms the live model mappings.

```
$env:CONTENTUNDERSTANDING_ENDPOINT="https://<foundry-resource>.services.ai.azure.com"
```

```
# Only when required: $env:CONTENTUNDERSTANDING_KEY="<private-training-value>"
```

```
.\.venv\Scripts\python.exe ..\labs\resources\content_understanding_invoice.py --check-config
```

3. Run prebuilt-invoice analysis and preserve the raw JSON and markdown locally. If the analyzer is unavailable, copy the named sanitized rejoin result and continue at source validation.

```
.\.venv\Scripts\python.exe ..\labs\resources\content_understanding_invoice.py --live --out evidence\invoice-result.json --markdown evidence\invoice-result.md
```

```
# Rejoin: Copy-Item ..\labs\resources\content-understanding-invoice-rejoin.json evidence\invoice-result-sanitized.json
```

4. Download the Microsoft synthetic invoice to work/, render or open page 1 locally, and locate VendorName, InvoiceId, InvoiceDate, SubTotal, TotalTax, InvoiceTotal and Items in the source and result.

```
Invoke-WebRequest 'https://raw.githubusercontent.com/Azure-Samples/azure-ai-content-understanding-assets/main/document/invoice.pdf' -OutFile work\invoice.pdf
```

```
Start-Process work\invoice.pdf
```

5. Complete work/invoice-field-validation.csv with extracted value, source value, page/region evidence, confidence and disposition.

```
Disposition: accept | correct | review | missing
```

6. Check arithmetic consistency across line items, subtotal, tax and total. Record mismatches as validation defects rather than modifying the raw analyzer result.

`subtotal + tax = total; preserve currency and decimal precision`

7. On the live path, create the shareable sanitized result with the supplied command. On the rejoin path, retain the already-sanitized supplied file and do not run this command without a raw result.

`.\.venv\Scripts\python.exe ..\labs\resources\content_understanding_invoice.py --sanitize evidence\invoice-result.json --out evidence\invoice-result-sanitized.json`

8. Design work/custom-invoice-analyzer.md with three additional fields and explicit descriptions; do not deploy it in this time-box.

Suggested: `support_request_id | asset_tag | service_category`

9. Record latency, missing and low-confidence fields, review threshold and evidence links in the ADR, then update evidence/manifest.md.

A syntactically valid field can still be wrong; evidence review remains required.

Test it

The prebuilt analyzer returns document content, seven critical field groups are checked against source evidence, arithmetic is reconciled, and uncertain or missing values are routed to review rather than silently accepted.

Troubleshooting

- The SDK cannot import.: Reinstall azure-ai-contentunderstanding in the active virtual environment and verify the interpreter used by python.
- The operation returns unauthorized.: Verify endpoint resource, environment variable scope and the instructor-provided role or key; never hard-code the key.
- A field value exists but has no trustworthy evidence.: Mark it review and preserve the raw content, page and field path for a person.

Challenge

Add a validation rule that detects duplicate invoice IDs across two sanitized results without storing the original documents.

Reflection

Which extracted fields can be accepted automatically, and what evidence threshold justifies that choice?

Note: Checkpoint: Save sanitized invoice evidence under C926-labs/evidence, editable validation/design files under work/, and update the ADR and evidence/manifest.md. Lab 9 indexes clean representations and tests grounded retrieval.

Note: Full commands and screenshots are in labs/lab-08-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Lab 9 — Build and Verify a Hybrid Grounding Pipeline

Learning outcome: LO6 - configure semantic, hybrid and vector retrieval, connect evidence to an agent workflow and verify retrieval and grounding quality..

Goal: Query an instructor-prepared Azure AI Search index with keyword, vector and hybrid modes, compare relevance, and pass compact cited evidence to the Northstar grounded app.

Duration and prerequisites

90 minutes

- Complete Lab 8 and retain its sanitized markdown and structured fields.
- Confirm the prepared index, vectorizer and semantic configuration, plus Search Index Data Reader on the prepared index or search-service scope.
- Set AZURE_SEARCH_ENDPOINT and AZURE_SEARCH_INDEX; use SEARCH_API_KEY only if the training environment cannot use Entra ID.

What you'll build

A retrieval comparison report, hybrid-query result, grounded response with source IDs and a production monitoring checklist. (Tools: Python, Azure AI Search REST API, Microsoft Foundry Responses API, labs/resources/hybrid_search.py.)

Step-by-step

1. Copy the index contract, sample documents and editable comparison template into work/, then identify the key, content, vector, source, freshness and permission-filter fields.

```
Copy-Item ..\labs\resources\search-index-contract.json work\search-index-contract.json
```

```
Copy-Item ..\labs\resources\search-documents.json work\search-documents.json
```

```
Copy-Item ..\labs\resources\retrieval-comparison.csv work\retrieval-comparison.csv
```

2. Run offline mode first. Confirm both the operations-only restricted document and the same-category restricted device document are excluded for learner scope.

```
.\.venv\Scripts\python.exe ..\labs\resources\hybrid_search.py --offline --query "replacement laptop evidence" --out evidence\search-offline.json
```

Expected absent: NS-RESTRICTED-001 and NS-RESTRICTED-DEVICE-002

3. Run a keyword query and record top source IDs, ranks and exact-term strengths; if unavailable, copy the exact keyword rejoin report.

```
.\venv\Scripts\python.exe ..\labs\resources\hybrid_search.py --mode keyword --  
query "replacement laptop evidence" --out evidence\search-keyword.json
```

```
# Rejoin: Copy-Item ..\labs\resources\search-keyword-rejoin.json evidence\search-  
keyword.json
```

4. Run a vector query using integrated text vectorization and record semantic matches that lack exact words; if unavailable, copy the exact vector rejoin report.

```
.\venv\Scripts\python.exe ..\labs\resources\hybrid_search.py --mode vector --  
query "proof required when a work computer must be exchanged" --out  
evidence\search-vector.json
```

```
# Rejoin: Copy-Item ..\labs\resources\search-vector-rejoin.json evidence\search-  
vector.json
```

5. Run a hybrid query with semantic ranking when supported. If the service is unavailable, copy the named answerable and unsupported rejoin reports into the exact downstream filenames.

```
.\venv\Scripts\python.exe ..\labs\resources\hybrid_search.py --mode hybrid --  
semantic --query "proof required for urgent laptop exchange" --out  
evidence\search-hybrid.json
```

```
# Rejoin: Copy-Item ..\labs\resources\search-hybrid-rejoin.json evidence\search-  
hybrid.json
```

```
# Rejoin: Copy-Item ..\labs\resources\search-unsupported-rejoin.json  
evidence\search-unsupported.json
```

6. Verify the combined category and learner-scope filter separately from relevance using the same-category restricted device document.

```
Filter: category eq 'device-support' and access_scope eq 'learner'
```

```
Expected excluded source: NS-RESTRICTED-DEVICE-002
```

7. Complete work/retrieval-comparison.csv for precision, coverage, empty results, freshness, permission filtering and latency across all three modes.

Do not compare scores from unlike ranking stages as if they shared one scale.

8. Run the unsupported retrieval unless its rejoin file exists, then pass both evidence files to the grounded app. The unsupported command fails unless supported=false with no citations.

```
if (-not (Test-Path evidence\search-unsupported.json)) {  
.\venv\Scripts\python.exe ..\labs\resources\hybrid_search.py --mode keyword --  
query "home renovation reimbursement" --out evidence\search-unsupported.json }  
}
```

```
.\.venv\Scripts\python.exe ..\labs\resources\grounded_app.py --evidence
evidence\search-hybrid.json --query "What proof should I attach for an urgent
laptop exchange?" --live --out evidence\final-grounded.json
```

```
.\.venv\Scripts\python.exe ..\labs\resources\grounded_app.py --evidence
evidence\search-unsupported.json --query "Can Northstar reimburse a home
renovation?" --live --expect-unsupported --out evidence\final-unsupported.json
```

9. Define monitoring for ingestion failures, document count, index freshness, empty retrieval, relevance, citation coverage, groundedness, latency, tokens, safety events and access-filter violations.

For each signal: query or view | threshold | owner | response

10. Update the ADR with the query pattern, filter, evidence contract, quality results, limitations and rollback path, then complete evidence/manifest.md.

Final checkpoint: architecture -> app -> agent -> multimodal -> text -> extraction
-> retrieval -> operations

Test it

Keyword, vector and hybrid reports contain source IDs and latency; the permission filter excludes restricted content; the final answer cites only retrieved sources; and the monitoring checklist covers ingestion, retrieval, generation, security and operations.

Troubleshooting

- The vector query reports no vectorizer.: Use the instructor-provided index that has integrated vectorization or switch to the documented rejoin result; do not invent embedding dimensions.
- Hybrid results are worse than keyword results.: Inspect query wording, k, fields, filters and semantic configuration; hybrid is a method to evaluate, not a guaranteed win.
- A restricted source appears.: Stop the workflow, preserve the trace, verify filter construction and index metadata, and do not pass the evidence to generation.

Challenge

Add a freshness test that inserts a later synthetic policy version and verifies the old source remains auditable but is not presented as current.

Reflection

Which retrieval and grounding signals would allow you to distinguish a stale index from a weak generation prompt?

Note: Checkpoint: Save all search and grounded reports under evidence/, the comparison under work/, and complete the ADR plus evidence/manifest.md. The Northstar course project is ready for final recap.

Note: Full commands and screenshots are in labs/lab-09-*.md. Use only the synthetic Northstar data supplied with this course. Keep credentials in managed identities, environment variables or approved secret stores, never in prompts, lab files or source control.

Wrap-Up - From Prototype to Operated AI Service

A production Azure AI solution remains trustworthy only while its data, models, tools, identities and owners remain current.

Before release

- Review task boundaries, identities, network paths and consequential tool controls.
- Run representative quality, retrieval, safety and failure-path checks.
- Confirm trace collection, sensitive-data handling, support ownership and rollback evidence.

After release

- Monitor user outcomes, retrieval health, groundedness, tool reliability, latency, tokens and safety signals.
- Triage evidence before changing data, prompts, models, tools or capacity.
- Retire stale sources, unused deployments, orphaned agents and unowned connections.

Next Steps

- Rebuild the Northstar solution in a separate development project using your own approved sandbox resources.
- Replace one synthetic integration with an authorized organisational dataset and define its identity and freshness controls.
- Convert the course quality rubric, trace review and release checklist into reusable engineering templates.
- Review the current Microsoft Learn AI-103 study guide and product documentation before implementing features in a live environment.

Glossary

- **Agent** — A model-directed system with instructions, state and tools for a bounded outcome.
- **Content Understanding** — A Foundry Tool that extracts multimodal content, structure and fields through analyzers.

- **Embedding** — A numeric representation used to compare semantic similarity.
- **Foundry project** — A project boundary for models, agents, connections, data, evaluations and observability.
- **Groundedness** — The extent to which a response is supported by the provided context or retrieved evidence.
- **Hybrid search** — Keyword and vector search executed together, with results fused into one ranking.
- **Managed identity** — An Azure-managed workload identity that avoids application-managed credentials.
- **RAG** — Retrieval-augmented generation: retrieve evidence, then generate an answer constrained by it.
- **Semantic ranker** — A language-understanding stage that reranks search results for relevance.
- **Tool schema** — The typed name, description, inputs and outputs an agent uses to select and call a capability.
- **Trace** — An ordered record of model, retrieval, tool and application spans for one interaction.
- **Vector search** — Similarity search over embedding vectors rather than exact terms alone.

Official Reference Sources

Product experiences change over time. Use these authoritative sources to verify current navigation, prerequisites and feature status.

- AI-103 study guide — <https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/ai-103>
- Azure AI Projects Python library — <https://learn.microsoft.com/en-us/python/api/overview/azure/ai-projects-readme?view=azure-python>
- Microsoft Foundry quickstarts — <https://learn.microsoft.com/en-us/azure/foundry/quickstarts/quickstarts>
- Responses API agents — <https://learn.microsoft.com/en-us/azure/foundry/agents/quickstarts/responses-api>
- Function tools and approval — <https://learn.microsoft.com/en-us/agent-framework/agents/tools/tool-approval>
- Agent tracing — <https://learn.microsoft.com/en-us/azure/foundry/observability/how-to/trace-agent-framework>
- Generative AI evaluation — <https://learn.microsoft.com/en-us/azure/ai-studio/how-to/evaluate-generative-ai-app>
- RAG evaluators — <https://learn.microsoft.com/en-us/azure/foundry/concepts/evaluation-evaluators/rag-evaluators>
- Azure AI Search vector quickstart — <https://learn.microsoft.com/en-us/azure/search/search-get-started-vector?pivots=python>

- Azure AI Search hybrid queries — <https://learn.microsoft.com/en-us/azure/search/hybrid-search-how-to-query>
- Content Understanding quickstart — <https://learn.microsoft.com/en-us/azure/ai-services/content-understanding/quickstart/use-rest-api>
- Content Understanding analyzer reference — <https://learn.microsoft.com/en-us/azure/ai-services/content-understanding/concepts/analyzer-reference>
- Speech to text quickstart — <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/get-started-speech-to-text>
- Translator SDK quickstart — <https://learn.microsoft.com/en-us/azure/ai-services/translator/text-translation/quickstart/client-library-sdk>
- Image generation tool — <https://learn.microsoft.com/en-us/azure/foundry/agents/how-to/tools/image-generation>